



Module 3:

Basic Machine Learning

Algorithms

By Dr. Shagun Sharma

Contents

- Basic Machine Learning Algorithms
- Association Rule mining – Linear Regression- Logistic Regression – Classifiers – k-Nearest Neighbors (k-NN),
- k-means -Decision tree – Naive Bayes- Ensemble Methods – Random Forest.
- Feature Generation and Feature Selection – Feature Selection algorithms –
- Filters; Wrappers; Decision Trees; Random Forests.

01

Basics of Machine Learning

Preface of Machine Learning

Machine Learning

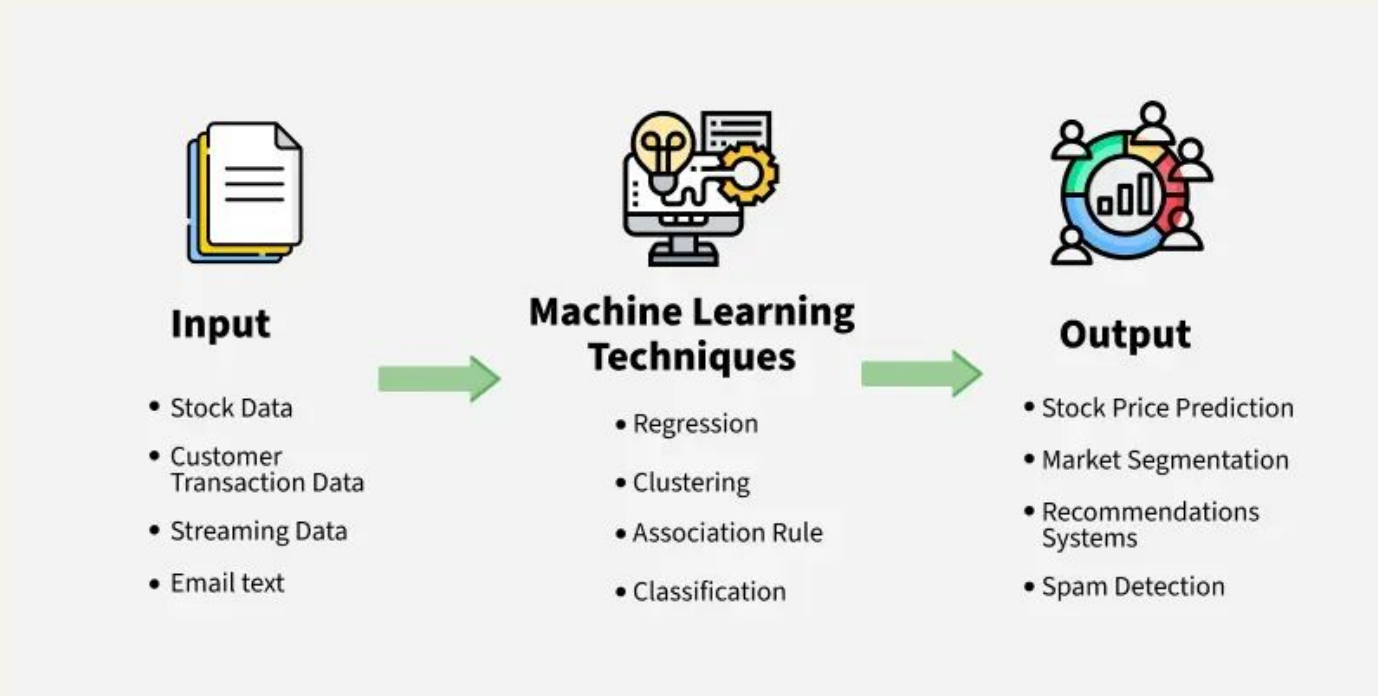
- Machine Learning (ML) is a field of computer science that focuses on building systems that can learn from data and improve their performance over time without being explicitly programmed. Instead of writing rules manually, we feed examples to a machine and let it learn patterns automatically.

OR

- Machine learning (ML) is a branch of artificial intelligence (AI) focused on enabling computers and machines to imitate the way that humans learn, to perform tasks autonomously, and to improve their performance and accuracy through experience and exposure to more data.

Preface of Machine Learning

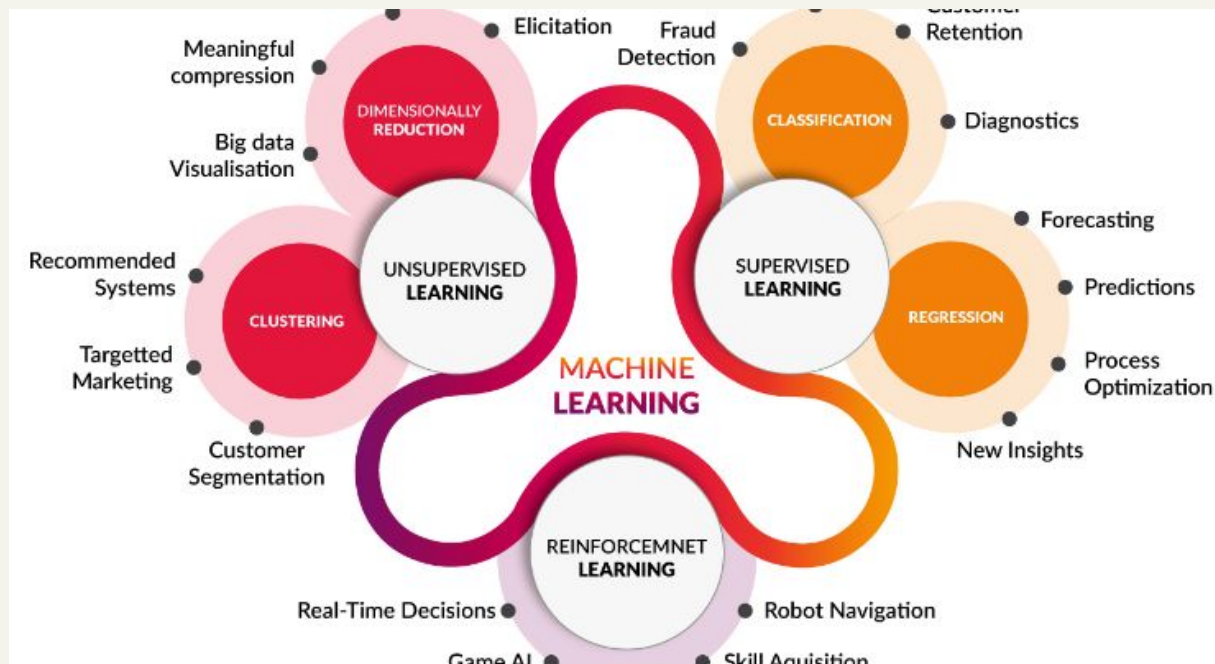
Process of Machine Learning



Types of Machine Learning

Types of Learning

Types of Machine Learning



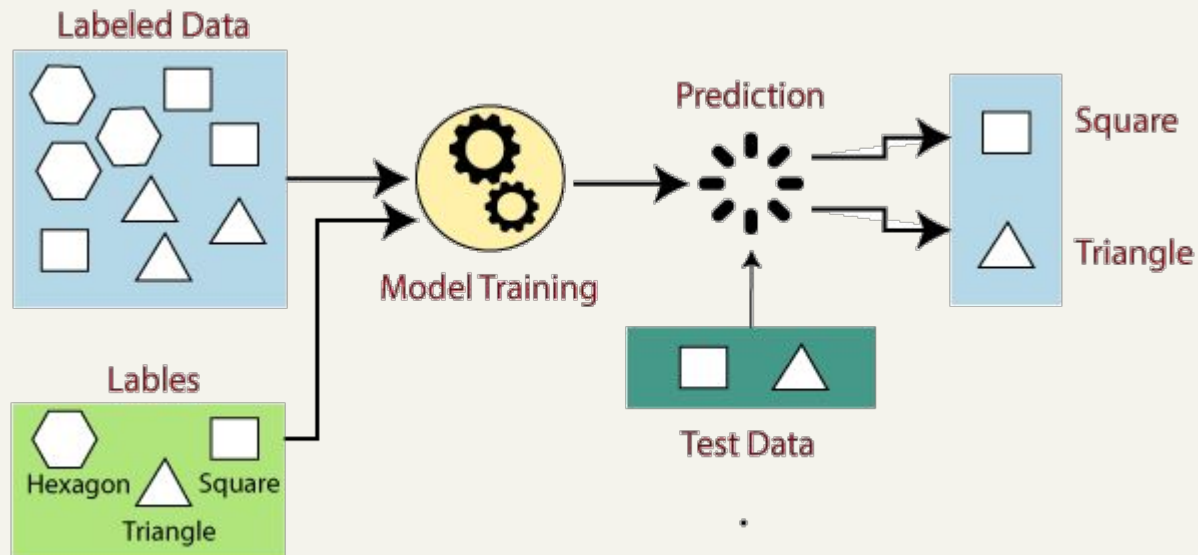
Types of Machine Learning

Supervised Machine Learning

- Supervised machine learning is a type of machine learning where an algorithm learns from a labeled dataset to make predictions or classifications. The algorithm is trained on data that includes both input features and the corresponding correct output, allowing it to learn the relationship between them and apply that knowledge to new, unseen data.

Types of Machine Learning

Supervised Machine Learning



Types of Machine Learning

Supervised Machine Learning Algorithms

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forests
- Support Vector Machines (SVM)
- K-Nearest Neighbors (k-NN)
- Neural Networks

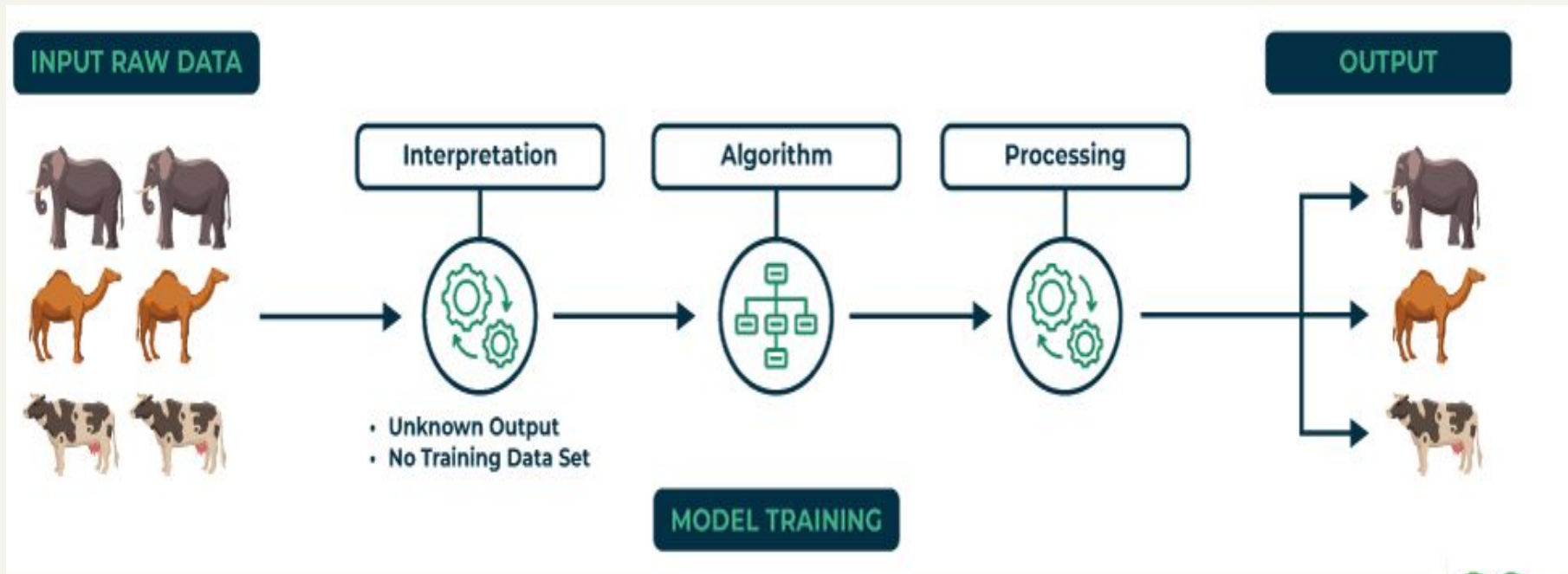
Types of Machine Learning

Unsupervised Machine Learning

- Unsupervised machine learning is a type of machine learning where algorithms learn from unlabeled data without explicit guidance or labeled outputs. The goal is to discover hidden patterns, relationships, and structures within the data by exploring it. Unlike supervised learning, there are no correct answers or pre-defined categories provided to the algorithm.

Types of Machine Learning

Unsupervised Machine Learning



Types of Machine Learning

Unsupervised Machine Learning Algorithms

- K-means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- Apriori Algorithm (for association rules)
- Autoencoders

Types of Machine Learning

Reinforcement Learning

- Reinforcement learning (RL) is a machine learning paradigm where an agent learns to make decisions by interacting with an environment to maximize a cumulative reward. It's a trial-and-error approach where the agent explores different actions, receives feedback in the form of rewards or penalties, and adjusts its behavior to achieve the best long-term outcome.

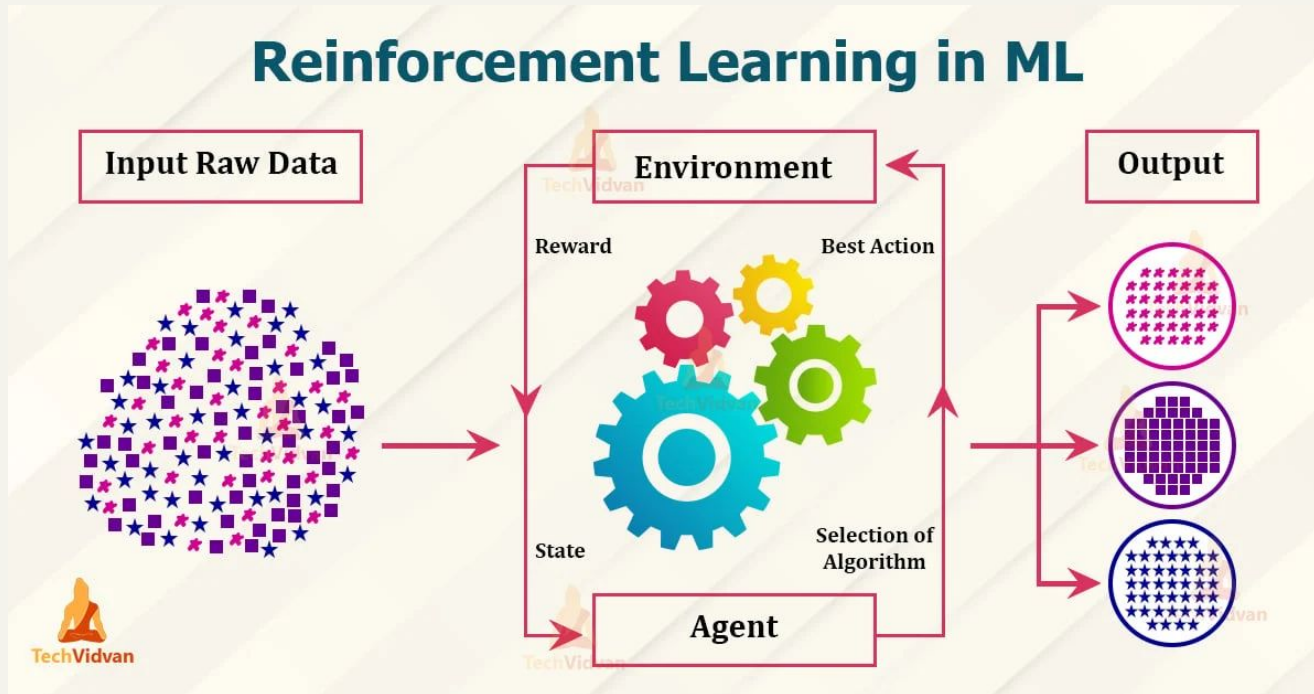
Types of Machine Learning

Concept of reinforcement Learning: How it Works

- **Exploration:** The agent starts by exploring the environment, trying different actions.
- **Feedback:** The agent receives feedback (rewards or penalties) based on its actions.
- **Learning:** The agent adjusts its policy to favor actions that lead to higher rewards.
- **Iteration:** This process repeats, allowing the agent to gradually learn the optimal policy for maximizing its cumulative reward.

Types of Machine Learning

Reinforcement Learning



02

Association Rule Mining

Association Rule Mining

For rule based modeling

- Association rule mining is a Machine Learning technique used to uncover hidden "if-then" relationships and frequent patterns between variables in large datasets. Commonly used in market basket analysis, it identifies items frequently purchased together, such as "if a customer buys bread, they are 80% likely to also buy butter".

Association Rule Mining

- Association rules are a fundamental concept used to find relationships, correlations or patterns within large sets of data items. They describe how often itemsets occur together in transactions and express implications of the form:

$$X \rightarrow Y$$
$$\{\text{Bread, Butter}\} \rightarrow \{\text{Milk}\}$$

Key Components

- Antecedent (X): The "if" part representing one or more items found in transactions.
- Consequent (Y): The "then" part, representing the items likely to be purchased when antecedent items appear.

Association Rule Mining

- Rules are evaluated based on metrics that quantify their strength and usefulness:

- Rule Evaluation Metrics**

1. Support: Fraction of transactions containing the itemsets in both X and Y .

$$\text{Support}(X \rightarrow Y) = \frac{\text{Number of transactions with } (X \cup Y)}{\text{Total number of transactions}}$$

2. Confidence: Probability that transactions with X also include Y .

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Association Rule Mining

- **Rule Evaluation Metrics**

3. Lift: The ratio of observed support to that expected if X and Y were independent.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

Lift > 1 implies a positive association — items occur together more than expected.

Lift = 1 implies independence.

Lift < 1 implies a negative association.

Example of Association Rule Mining

A supermarket records the following transactions:

Assume:

- Minimum Support = 60%
- Minimum Confidence = 70%

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Assume that the minimum support is 60% and the minimum confidence is 70%. Using the Apriori algorithm, find all frequent itemsets. Generate all possible association rules from the frequent itemsets, calculate the support and confidence of each rule, and identify the strong association rules that satisfy both the minimum support and minimum confidence thresholds.

Example of Association Rule Mining

Calculate Support of Each Item

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

$$\text{Support}(X \rightarrow Y) = \frac{\text{Number of transactions with } (X \cup Y)}{\text{Total number of transactions}}$$

Item	Count	Support
Milk	4	4/5 = 80%
Bread	4	4/5 = 80%
Butter	4	4/5 = 80%

All items satisfy **min support (60%)**

Example of Association Rule Mining

Step 3:
Generate 2-Itemsets

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Itemset	Count	Support
Milk, Bread	3	$3/5 = 60\%$
Milk, Butter	3	$3/5 = 60\%$
Bread, Butter	3	$3/5 = 60\%$

All satisfy minimum support (60%)

Example of Association Rule Mining

Step 4: Generate 3-Itemset

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Itemset	Count	Support
Milk, Bread, Butter	2	$2/5 = 40\%$

40% < 60% → Not frequent
So we stop here.

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 1: Milk $\rightarrow$ Bread

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Confidence}(\text{Milk} \rightarrow \text{Bread}) = \frac{3/5}{4/5} = 3/4 = 75$$

Support = 60%

Confidence = 75%

Rule Accepted

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 2: Bread $\rightarrow$ Milk

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Support = 60%
Confidence = 75%

$$\text{Confidence}(\text{Bread} \rightarrow \text{Milk}) = \frac{3/5}{4/5} = 75$$

Rule Accepted

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 3: Milk $\rightarrow$ Butter

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Confidence} = 3/4 = 75$$

Support = 60%

Confidence = 75%

Rule Accepted

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 4: Butter $\rightarrow$ Milk

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Confidence} = 3/4 = 75$$

Support = 60%

Confidence = 75%

Rule Accepted

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 5: Bread $\rightarrow$ Butter

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Confidence} = 3/4 = 75$$

Support = 60%

Confidence = 75%

Rule Accepted

Example of Association Rule Mining

Step 5: Generate Association Rules
Now generate rules from frequent 2-itemsets.

Transaction ID	Items Purchased
T1	Milk, Bread, Butter
T2	Bread, Butter
T3	Milk, Bread
T4	Milk, Butter
T5	Milk, Bread, Butter

Rule 6: Butter $\rightarrow$ Bread

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

$$\text{Confidence} = 3/4 = 75$$

Support = 60%

Confidence = 75%

Rule Accepted

Association Rule Mining

Final Strong Association Rules

- All rules below satisfy:
- Support $\geq 60\%$
- Confidence $\geq 70\%$
- Milk $\rightarrow$ Bread
- Bread $\rightarrow$ Milk
- Milk $\rightarrow$ Butter
- Butter $\rightarrow$ Milk
- Bread $\rightarrow$ Butter
- Butter $\rightarrow$ Bread

Example:

Milk $\rightarrow$ Bread (75%) means:

75% of customers who buy Milk also buy Bread.

Solved Example:

http://www.saedsayad.com/association_rules.htm

Solve it Yourself!

An online electronics store records the following customer purchase transactions during a festive sale:

T1:	{Laptop,	Mouse,	Keyboard}
T2:	{Laptop,	Mouse}	
T3:	{Smartphone,	Earphones,	Charger}
T4:	{Laptop,	Keyboard,	Mouse}
T5:	{Smartphone,	Charger}	
T6:	{Laptop,	Mouse,	Headphones}
T7:	{Smartphone,	Earphones}	
T8:	{Laptop, Mouse, Keyboard, Headphones}		

Using the **Apriori algorithm**, generate all frequent itemsets with a minimum support of 37.5%. Then, derive all strong association rules with a minimum confidence of 75%. Calculate support and confidence for each generated rule and identify the rule with the highest confidence.

03

Regression: Linear Regression, Logistic Regression

Regression

Regression is a supervised machine learning technique used to predict a continuous output (numeric value) based on one or more input variables (features).

Purpose of Regression

- To model the relationship between the independent variable(s) (inputs) and a dependent variable (output).
- To forecast or estimate real-world quantities like:

House prices

Temperature

Stock values

Medical costs

Exam scores

Linear Regression

Linear Regression is a statistical method used to model the linear relationship between a single independent variable x and a continuous dependent variable y .

$$y = \beta_0 + \beta_1 x + \varepsilon$$

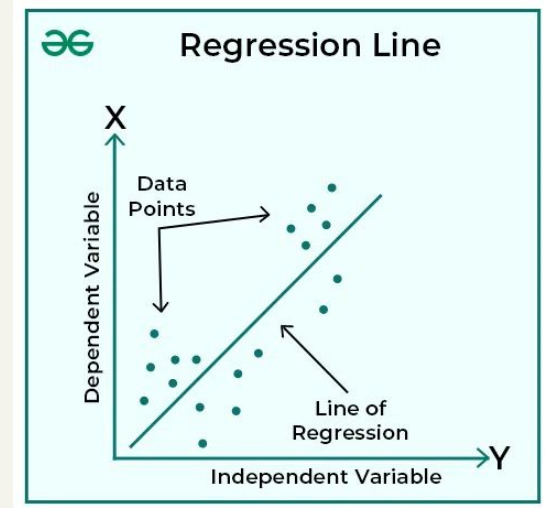
Where

- y = dependent variable (target)
- x = independent variable (feature)
- β_0 = intercept (bias)- the predicted value of the dependent variable when all independent variables are zero
- β_1 = slope of the line (coefficient)(represents the average change in the dependent variable for each one-unit increase in the independent variable)
- ε = error term (difference between predicted and actual)

Linear Regression

A linear regression graph visually represents the relationship between two variables, specifically an independent variable (plotted on the x-axis) and a dependent variable (plotted on the y-axis), and allows for predictions to be made. This relationship is depicted by a straight line, often called the "line of best fit", drawn through the data points on a scatterplot.

Purpose: To find the line (function) that minimizes the error between predicted and actual values, typically using Least Squares Error:

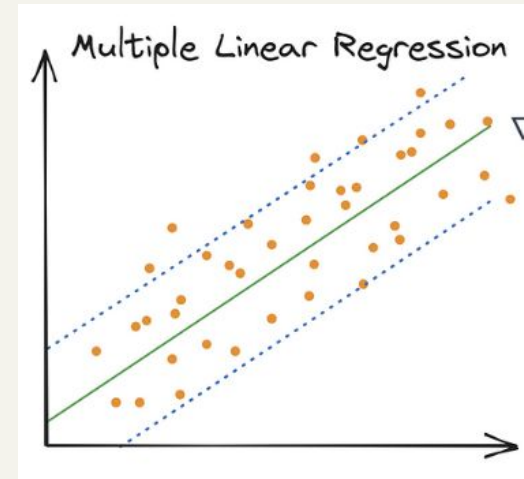


$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Multiple Linear Regression

Multiple Linear Regression models the linear relationship between two or more independent variables and a single continuous dependent variable.

Purpose: To fit a hyperplane in higher-dimensional space that best explains the variation in the target.

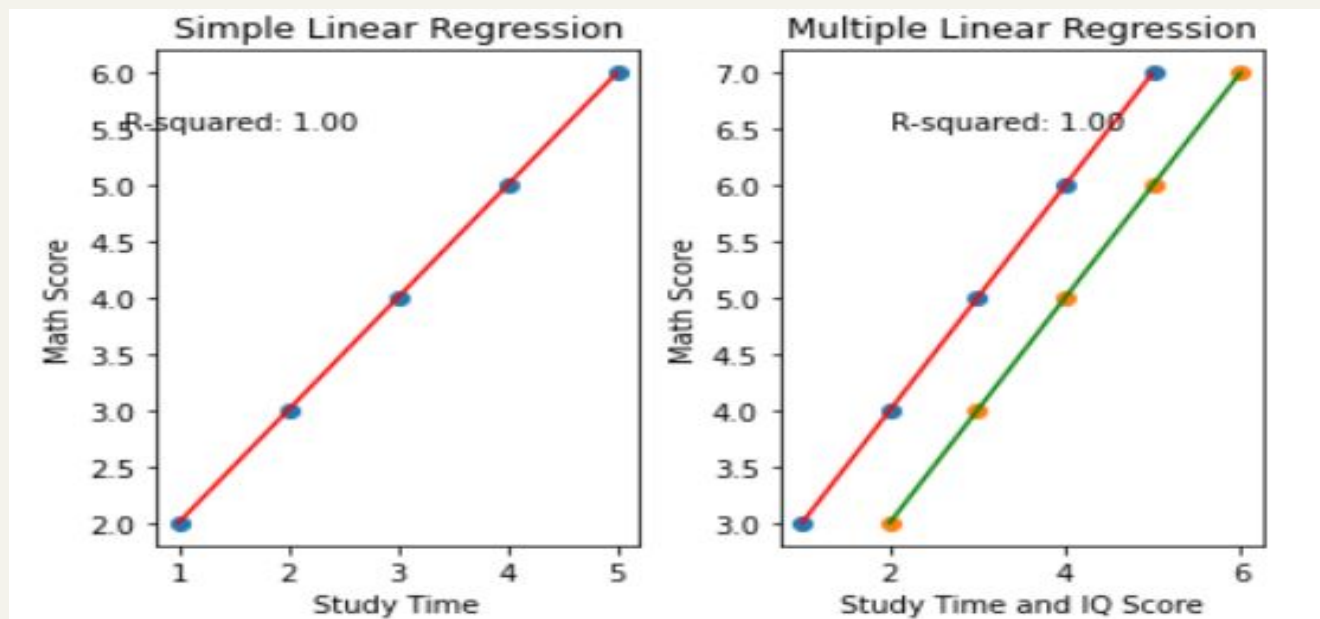


$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

Difference between Linear and Multiple Linear Regression

The main difference between linear and multilinear regression lies in the number of independent variables used to predict a dependent variable. Linear regression uses only one independent variable, while multilinear regression uses two or more.

Where, R-squared is a statistical measure that indicates how well a regression model fits the data



Difference between Linear and Multiple Linear Regression

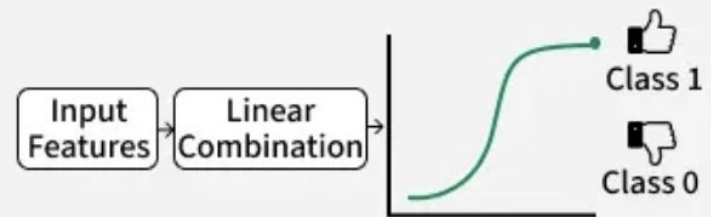
Feature	Linear Regression	Multiple Linear Regression
Number of input variables	One	Two or more
Equation form	$y = \beta_0 + \beta_1 x$	$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$
Geometric representation	Straight line	Hyperplane
Complexity	Simple	More complex
Use case example	Predict marks from study hours	Predict salary from age, experience, education

Logistic Regression

Logistic Regression is a supervised learning algorithm used for classification problems, especially binary classification (i.e., where the output is either 0 or 1, Yes or No, True or False).

Despite the name, it's actually a classification algorithm, not a regression one.

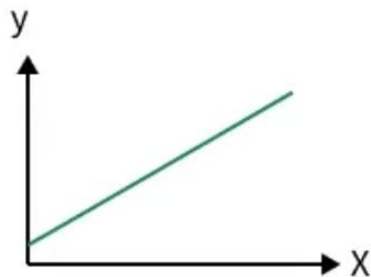
- Predicts the probability of a binary outcome (Yes/No, 0/1)
- Uses the sigmoid function to map inputs to probabilities (0 to 1)
- Ideal for classification tasks



Logistic Regression

Linear Regression

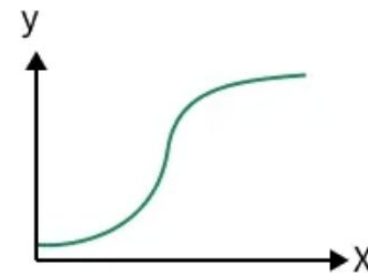
- Predicts continuous values
- Uses best-fit line
- Solves regression problems



vs

Logistic Regression

- Predicts categorical classes
- Uses sigmoid S-curve
- Solves classification problems



Logistic Regression

It uses linear equation + Sigmoid Activation function

Linear Equation:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Sigmoid Equation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Logistic Regression:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

Logistic Regression

Imagine trying to separate two classes (e.g., spam vs. not spam). The linear equation helps find a **line (or hyperplane)** that best separates the two. The sigmoid ensures that your output is interpretable as a **confidence level or probability** of being in a particular class.

Component	Purpose
Linear Equation	Maps inputs to a weighted sum — helps identify a decision boundary .
Sigmoid Function	Converts the output into a probability between 0 and 1, allowing for binary classification.

If $\sigma(z) \geq 0.5$, predict class 1.

If $\sigma(z) < 0.5$, predict class 0.

04

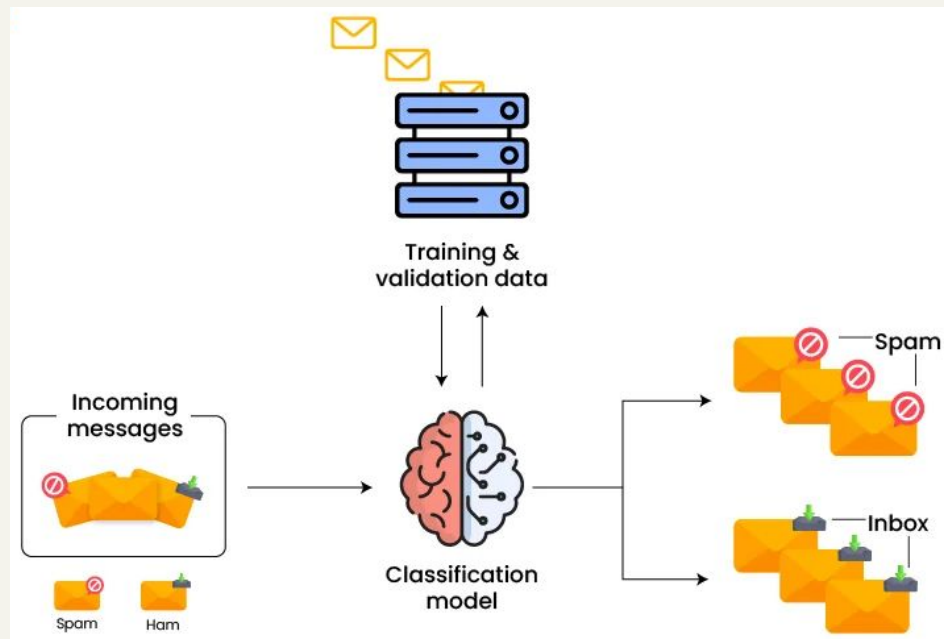
Classifiers

k-Nearest Neighbors (KNN)

Classifiers

ML Classifiers

- A classifier in machine learning is a supervised learning algorithm that assigns input data to predefined categories or classes based on labeled training data. By learning patterns, it predicts the category for new, unseen data, commonly used for tasks like spam detection, image recognition, and risk assessment.



Classifiers

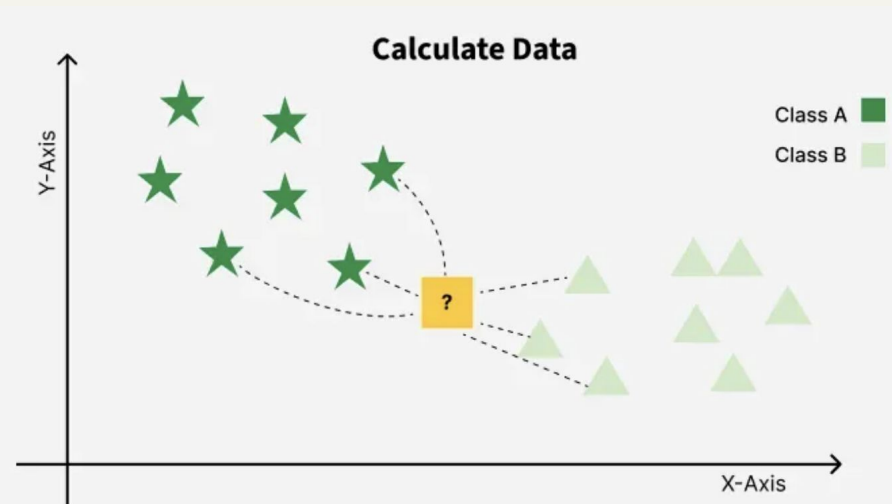
Supervised Machine Learning Algorithms

- Logistic Regression
- k-Nearest Neighbors (k-NN)
- Decision Tree
- Naive Bayes
- Random Forest
- Ensemble Methods

K-Nearest Neighbors

KNN

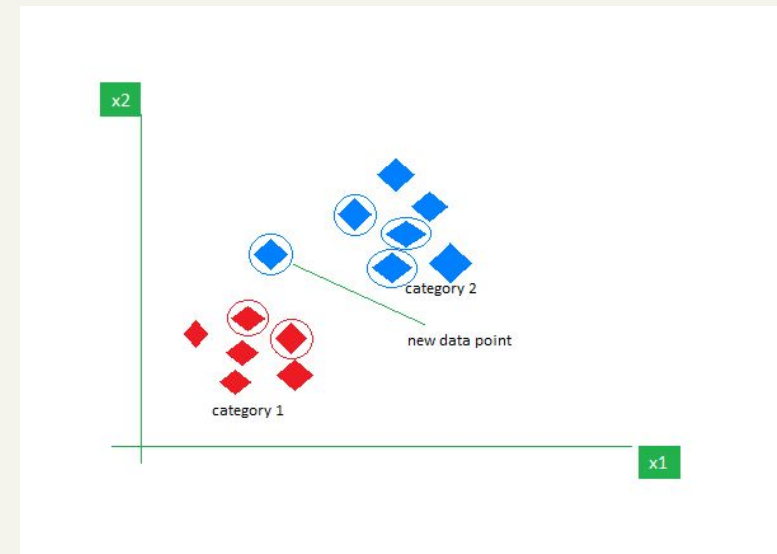
- K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression).



K-Nearest Neighbors

KNN

- K-Nearest Neighbors is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification it performs an action on the dataset.



K-Nearest Neighbors

KNN Example

Point	x	y	Class
P1	1	2	A
P2	2	3	A
P3	3	3	B
P4	5	5	B

You are given a new point: $Q=(3,4)$

and asked to classify it using $K = 3$ (i.e., use the 3 nearest neighbors).

K-Nearest Neighbors

KNN Example

Calculate Euclidean Distances from Q to all Points

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Nearest Neighbors

KNN Example

Sort the Distances

Point	Class	Distance to Q
P3	B	1.00
P2	A	1.41
P4	B	2.24
P1	A	2.83

K-Nearest Neighbors

KNN Example

Choose the $K = 3$ Nearest Neighbors

Nearest neighbors:

- **P3 (B)**
- **P2 (A)**
- **P4 (B)**

So the classes of the 3 neighbors are:

- **B, A, B**

Majority Voting

Count the classes:

- Class A: 1 vote
- Class B: **2 votes**

The point **Q = (3, 4)** is classified as **Class B** using **K = 3** in KNN.

K-Nearest Neighbors

Solve the following example by calculating the class for $Q=(3,2)$ and $K=3$

Point	x	y	Class
A	1	1	Red
B	2	1	Red
C	4	3	Blue
D	5	4	Blue
E	3	3	Red

K-Nearest Neighbors

Problem 2 - K Nearest Neighbor (10pts)

- Please use KNN with Euclidean distance to predict the label for data sample Strawberry with $K = 1, 3$ and 5 .
- Why KNN is called a lazy learner? What are the advantage and disadvantages of KNN?

Fruit	Sweetness	Sourness	Fruit Type
Lemon	1	9	Sour
Grapfruit	2	8	Sour
Orange	3	7	Sour
Cherry	6	4	Sweet
Banana	9	1	Sweet
Grapes	8	2	Sweet
Strawberry	5	5	?

05

Decision Tree Classifier

Decision Tree (ID3-Iterative Dichotomiser 3)

- A Decision Tree is a supervised machine learning algorithm used for classification and regression. It splits data into subsets based on feature values, using if-else conditions in a tree-like structure.
- Each internal node represents a decision on an attribute, each branch represents the outcome of the decision, and each leaf node represents a class label (in classification) or a value (in regression).

How Does the Tree Get Built?

- The tree chooses attributes that give the best split using Information Gain or Gini Index.

Decision Tree

Entropy (used in ID3 & optionally in classification): Entropy measures the impurity (disorder) of a dataset.

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

Where:

- p_i is the proportion of class i in dataset S .
- c is the number of classes.

Example of Entropy

If a dataset has 60% "Yes" and 40% "No":

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

$$\text{Entropy} = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) \approx 0.971$$

Decision Tree

Information Gain (IG) (used in ID3): Information Gain tells us which feature is most useful to classify the data.

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

Where:

- A : attribute being evaluated.
- S_v : subset of S where attribute A has value v .

Decision Tree

Gini Index (used in CART Classification Trees): Gini Index also measures impurity, but it's faster to compute than entropy.

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

Where:

- P_i : proportion of class i

Example of Gini Index

If 70% "Yes" and 30% "No":

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

$$\text{Gini} = 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) = 0.42$$

Decision Tree Example

Example Problem: Play Tennis Dataset

Day	Outlook	Temp	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes

Entropy calculation

Entropy measures impurity or disorder in the data.

$$Entropy(S) = -p_{Yes} \log_2(p_{Yes}) - p_{No} \log_2(p_{No})$$

- Total examples = 10

7 Yes

3 No

$$\begin{aligned} Entropy(S) &= - \left(\frac{7}{10} \log_2 \frac{7}{10} + \frac{3}{10} \log_2 \frac{3}{10} \right) = -(0.7 \cdot \log_2(0.7) + 0.3 \cdot \log_2(0.3)) \\ &= -(0.7 \cdot -0.514 + 0.3 \cdot -1.737) = 0.881 \end{aligned}$$

Information Gain for each attribute

1. Attribute: Outlook (Sunny, Overcast, Rain)

Outlook	Total	Yes	No
Sunny	4	1	3
Overcast	2	2	0
Rain	4	4	1

$$\text{Entropy}(\text{Sunny}) = -\frac{1}{4} \log_2\left(\frac{1}{4}\right) - \frac{3}{4} \log_2\left(\frac{3}{4}\right) \approx 0.811$$

$$\text{Entropy}(\text{Overcast}) = 0 \text{ (only Yes)}$$

$$\text{Entropy}(\text{Rain}) = -\frac{4}{5} \log_2\left(\frac{4}{5}\right) - \frac{1}{5} \log_2\left(\frac{1}{5}\right) \approx 0.721$$

$$\text{Gain}(S, \text{Outlook}) = \text{Entropy}(S) - \left[\frac{|S_{\text{Sunny}}|}{|S|} \cdot \text{Entropy}(S_{\text{Sunny}}) + \frac{|S_{\text{Overcast}}|}{|S|} \cdot \text{Entropy}(S_{\text{Overcast}}) + \frac{|S_{\text{Rain}}|}{|S|} \cdot \text{Entropy}(S_{\text{Rain}}) \right]$$

$$= 0.881 - \left(\frac{4}{10} \cdot 0.811 + \frac{2}{10} \cdot 0 + \frac{4}{10} \cdot 0.721 \right) = 0.881 - (0.324 + 0 + 0.288) = 0.269$$

Information Gain calculation

2. Attribute: Overcast → Days [3, 7]

All Yes → Entropy = 0

3. Outlook = Rain → Days [4, 5, 6, 10]

Day	PlayTennis
4	Yes
5	Yes
6	No
10	Yes

$$IG(S, A) = Entropy(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Yes = 3, No = 1

$$Entropy(Rain) = -\frac{3}{4} \log_2 \left(\frac{3}{4} \right) - \frac{1}{4} \log_2 \left(\frac{1}{4} \right) = 0.811$$

$$= -(0.25 \cdot -2) - (0.75 \cdot -0.415) = 0.5 + 0.311 = 0.811$$

Information Gain for each attribute

1. B) Temperature

Temp	Total	Yes	No
Hot	2	0	2
Mild	3	3	0
Cool	5	4	1

Entropy(Hot) = 0

Entropy(Mild) = 0

Entropy(Cool) = same as Rain above ≈ 0.721

$$Gain(Temp) = 0.881 - \left(\frac{2}{10} \cdot 0 + \frac{3}{10} \cdot 0 + \frac{5}{10} \cdot 0.721 \right) = 0.881 - 0.360 = 0.521$$

Information Gain for each attribute

1. C) Humidity

Humidity	Total	Yes	No
High	5	2	3
Normal	5	5	0

$$\text{Entropy}(\text{High}) = -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) \approx 0.971$$

$$\text{Entropy}(\text{Normal}) = 0$$

$$\text{Gain}(\text{Humidity}) = 0.881 - (0.5 \cdot 0.971 + 0.5 \cdot 0) = 0.881 - 0.485 = 0.396$$

Information Gain for each attribute

1. D) Wind

Wind	Total	Yes	No
Weak	6	5	1
Strong	4	2	2

$$\text{Entropy(Weak)} = -\frac{5}{6} \log_2\left(\frac{5}{6}\right) - \frac{1}{6} \log_2\left(\frac{1}{6}\right) \approx 0.650$$

$$\text{Entropy(Strong)} = 1.0$$

$$\text{Gain(Wind)} = 0.881 - (0.6 \cdot 0.650 + 0.4 \cdot 1.0) = 0.881 - (0.39 + 0.4) = 0.091$$

Calculating the root node

Attribute	Gain
Outlook	0.269
Temperature	0.521
Humidity	0.396
Wind	0.091

Temperature has the highest gain → make it the root node.

Question for Practice (DT)

Rule No.	Credit Score	Income	Existing Loan	Collateral	Action
1	Good	High	No	Yes	Approve
2	Good	High	Yes	Yes	Approve
3	Good	Low	No	Yes	Approve
4	Good	Low	Yes	No	Reject
5	Bad	High	No	Yes	Reject
6	Bad	High	Yes	No	Reject
7	Bad	Low	No	No	Reject
8	Bad	Low	Yes	Yes	Reject

Solution

There are 8 total samples:

- **Approve: 3**
- **Reject: 5**

$$P(\text{Approve}) = \frac{3}{8}, \quad P(\text{Reject}) = \frac{5}{8}$$

$$\text{Entropy}(S) = - \left(\frac{3}{8} \log_2 \frac{3}{8} + \frac{5}{8} \log_2 \frac{5}{8} \right)$$

$$= - (0.375 \cdot \log_2 0.375 + 0.625 \cdot \log_2 0.625) = - (0.375 \cdot -1.415 + 0.625 \cdot -0.678)$$

$$= 0.5306 + 0.4237 = \boxed{0.9543}$$

IG for credit score

Credit Score	Count	Approve	Reject
Good	4	3	1
Bad	4	0	4

Entropy(Good):

$$= - \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) = - (0.75 \cdot -0.415 + 0.25 \cdot -2) = 0.311 + 0.5 = 0.811$$

Entropy(Bad):

$$= - \left(\frac{0}{4} \log_2 \frac{0}{4} + \frac{4}{4} \log_2 \frac{4}{4} \right) = 0$$

$$IG(CreditScore) = Entropy(S) - \left(\frac{4}{8} \cdot 0.811 + \frac{4}{8} \cdot 0 \right) = 0.9543 - 0.4055 = \boxed{0.5488}$$

IG for Income

Income	Count	Approve	Reject
High	4	2	2
Low	4	1	3

Entropy(High): 1

Entropy(Low): 0.811

$$IG(Income) = 0.9543 - \left(\frac{4}{8} \cdot 1 + \frac{4}{8} \cdot 0.811 \right) = 0.9543 - (0.5 + 0.4055) = \boxed{0.0488}$$

IG for Existing Loan

Existing Loan	Count	Approve	Reject	Entropy
No	4	2	2	1.0
Yes	4	1	3	0.811

$$IG(ExistingLoan) = 0.9543 - (0.5 + 0.4055) = \boxed{0.0488}$$

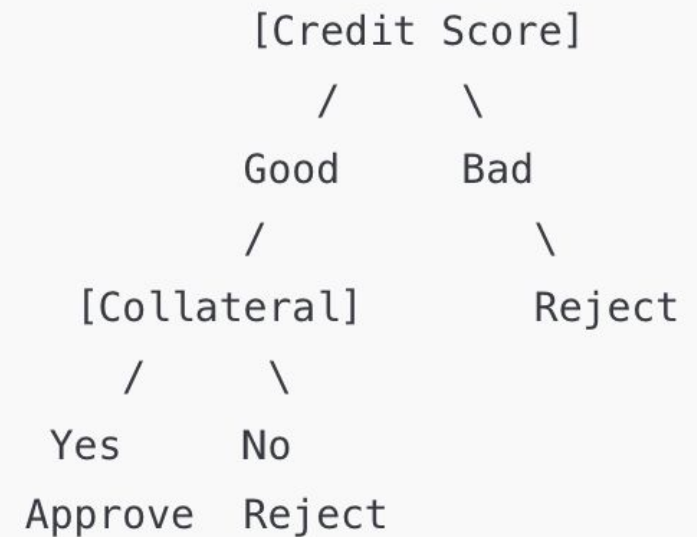
IG for Collateral

Collateral	Count	Approve	Reject	Entropy
Yes	5	3	2	0.971
No	3	0	3	0

$$IG(Collateral) = 0.9543 - \left(\frac{5}{8} \cdot 0.971 + \frac{3}{8} \cdot 0 \right) = 0.9543 - 0.607 = \boxed{0.3473}$$

IG for all

Attribute	Information Gain
Credit Score	0.5488 ✓ (best)
Collateral	0.3473
Income	0.0488
Existing Loan	0.0488



07

Ensemble Methods - Random Forest

Ensemble Learning Model: Combination Schemes

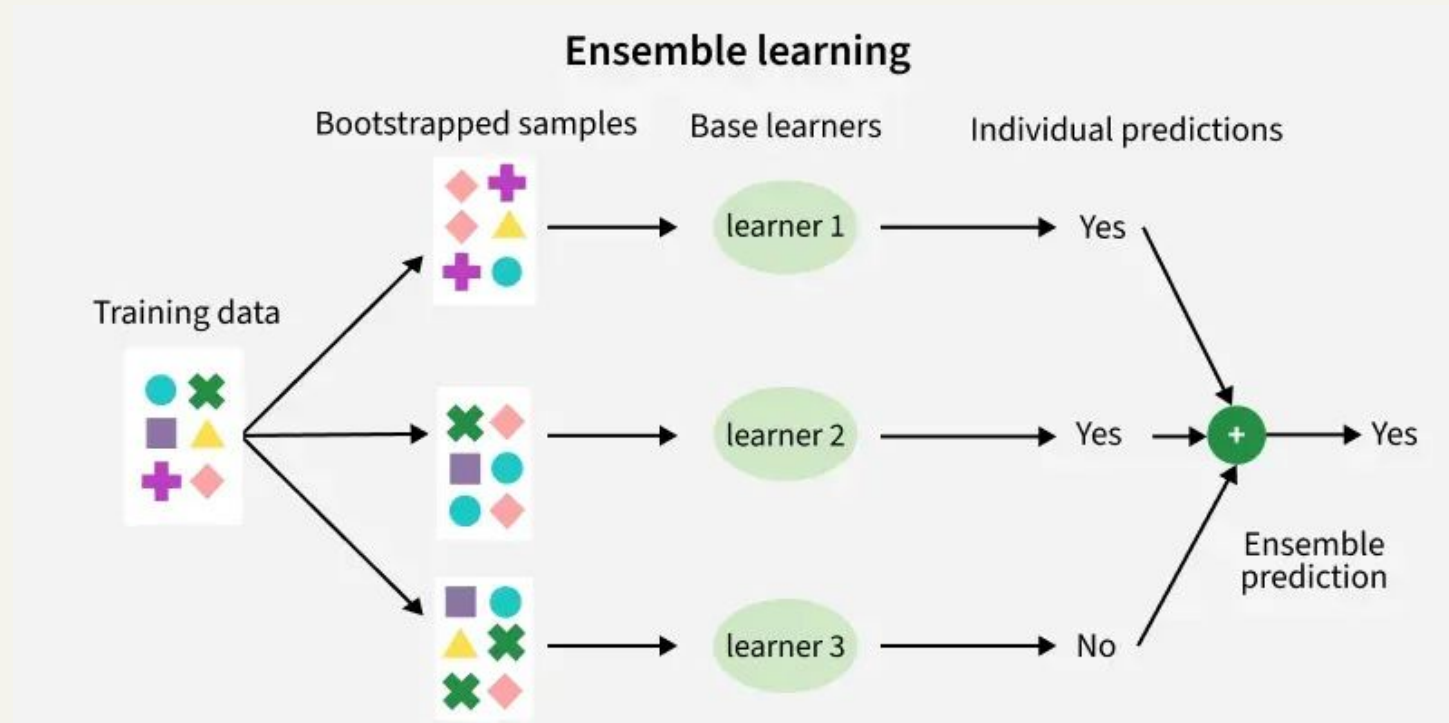
Ensemble Learning

- Ensemble learning is a method where we use many small models instead of just one. Each of these models may not be very strong on its own, but when we put their results together, we get a better and more accurate answer.

Example:

- It's like asking a group of people for advice instead of just one person — each one might be a little wrong, but together, they usually give a better answer.

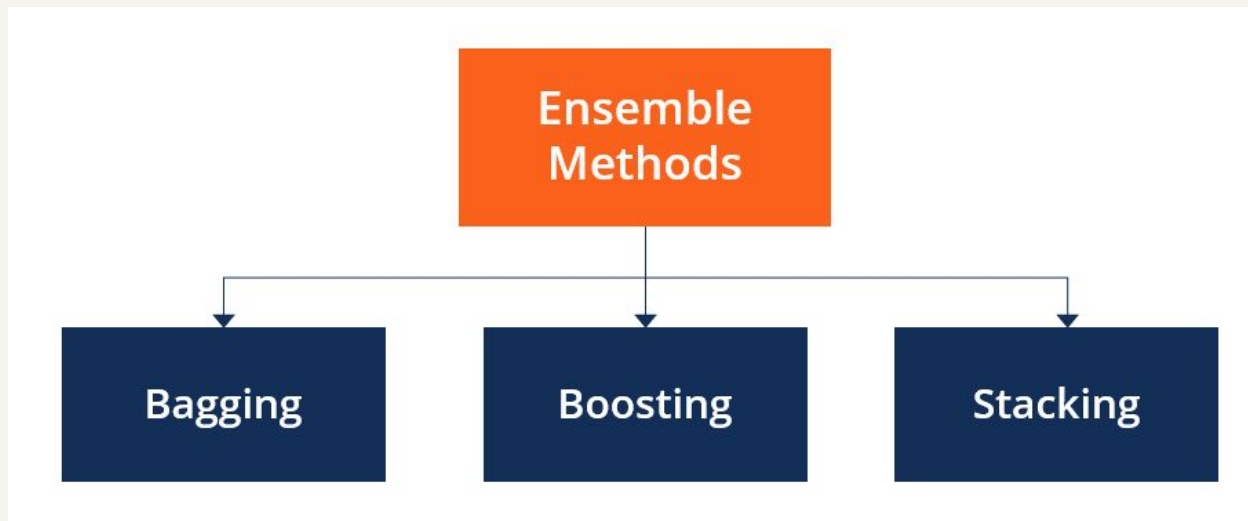
Ensemble Learning Model: Combination Schemes



Ensemble Learning Model: Combination Schemes

Types of Ensemble Learning

- There are three main types of ensemble methods:



Ensemble Learning Model: Combination Schemes

Bagging (Bootstrap Aggregating):

- Models are trained independently on different random subsets of the training data. Their results are then combined – usually by averaging (for regression) or voting (for classification). This helps reduce variance and prevents overfitting.

Boosting:

- Models are trained one after another. Each new model focuses on fixing the errors made by the previous ones. The final prediction is a weighted combination of all models, which helps reduce bias and improve accuracy.

Ensemble Learning Model: Combination Schemes

Stacking (Stacked Generalization):

- Multiple different models (often of different types) are trained, and their predictions are used as inputs to a final model, called a meta-model. The meta-model learns how to best combine the predictions of the base models, aiming for better performance than any individual model.

Ensemble Learning Model: Combination Schemes

Bagging

Bagging

Different Subset
of the Training Data



Base Models



Most Frequent or
Average Prediction



Prediction

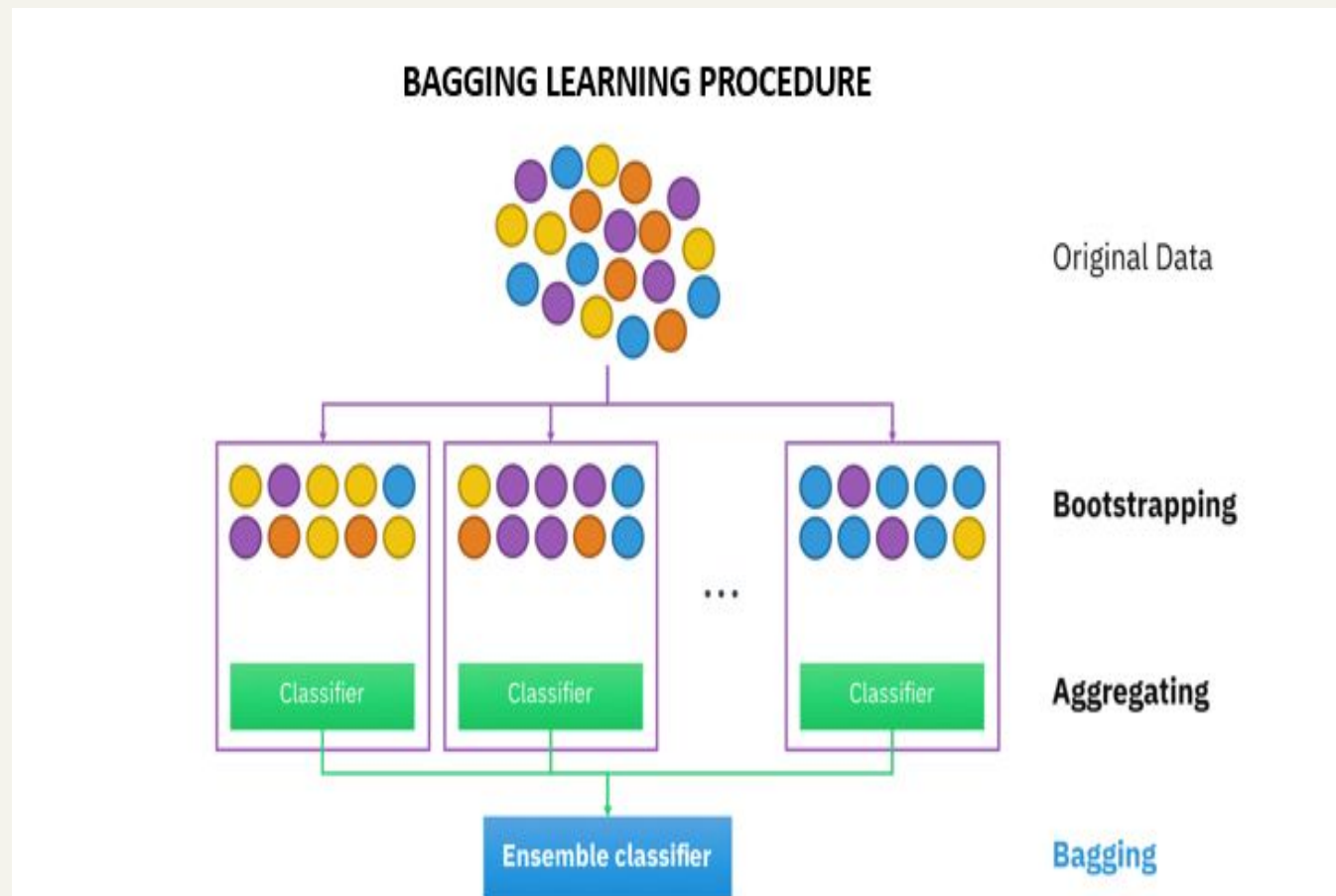
Ensemble Learning Model: Combination Schemes

Bagging

- It is an ensemble learning technique used to improve the stability, accuracy, and robustness of machine learning models — especially those that are high-variance.
- It combines predictions from multiple models (usually of the same type), each trained on a random subset of the data, to reduce overfitting and increase overall performance.

Ensemble Learning Model: Combination Schemes

Bagging



Ensemble Learning Model: Combination Schemes

How Aggregation Happens in Bagging?

Model 1 $\rightarrow$ Class A

Model 2 $\rightarrow$ Class B

Model 3 $\rightarrow$ Class A

Model 4 $\rightarrow$ Class A

Model 5 $\rightarrow$ Class B

So?

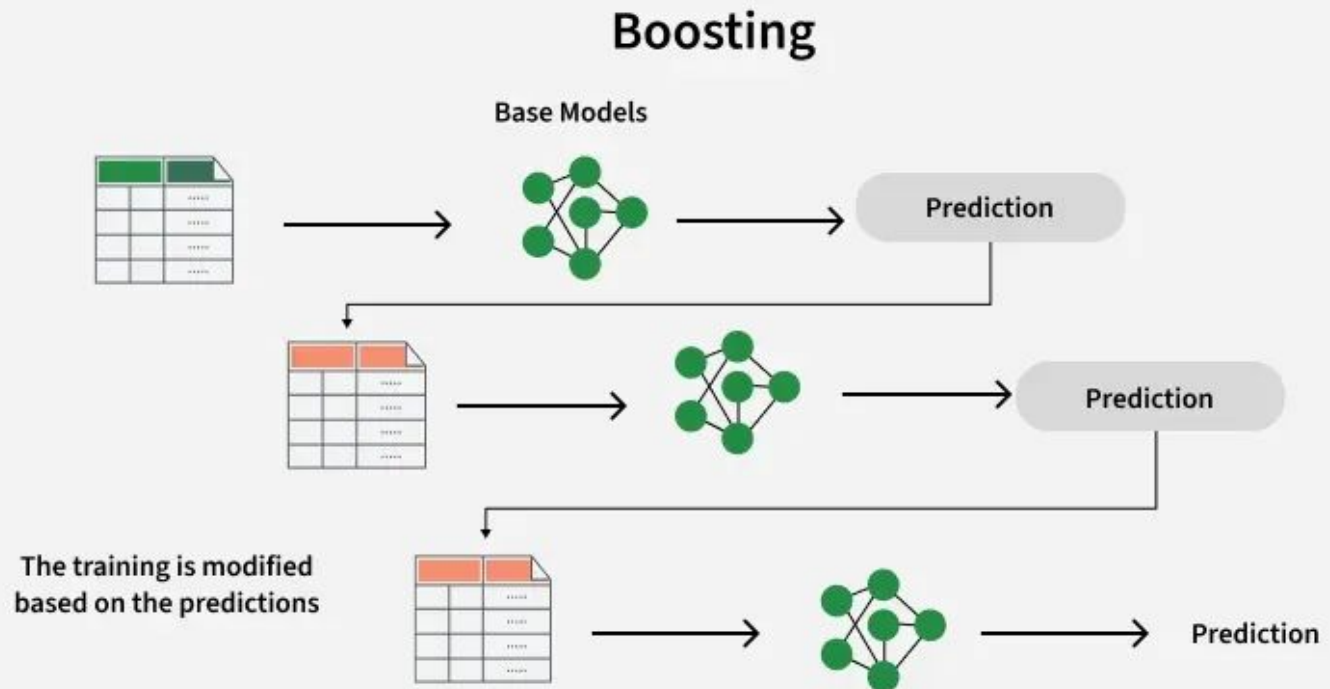
Class A $\rightarrow$ 3 votes

Class B $\rightarrow$ 2 votes

For regression it will be average.

Ensemble Learning Model: Combination Schemes

Boosting



Ensemble Learning Model: Combination Schemes

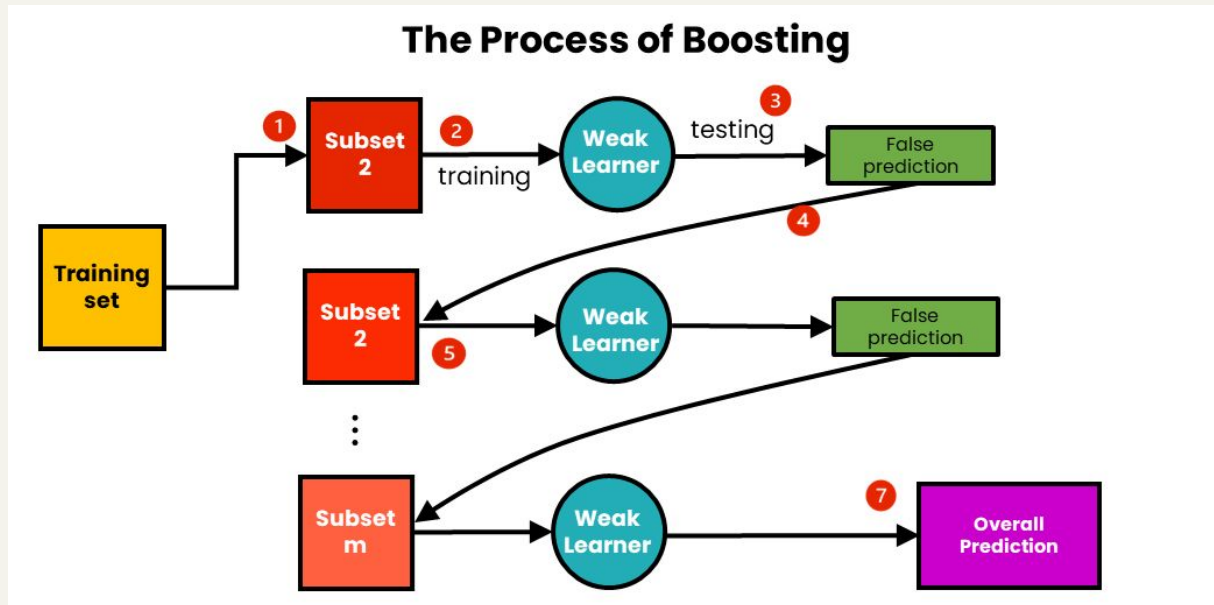
Boosting

- Boosting is an ensemble learning technique that builds a strong learner by sequentially combining multiple weak learners — each one correcting the errors made by the previous ones.
- Instead of training multiple models independently (like in bagging), boosting trains models sequentially, where each new model:

Focuses more on the data points that were misclassified (or poorly predicted) by the earlier models.

Ensemble Learning Model: Combination Schemes

Boosting



Ensemble Learning Model: Combination Schemes

How does Boosting work?

Boosting is an ensemble learning technique that aims to improve the accuracy and robustness of predictive models by combining the outputs of several weak learners into a single strong learner. A weak learner is typically a model that performs slightly better than random guessing, such as a shallow decision tree.

Unlike bagging, where models are trained independently and in parallel, boosting builds models sequentially, with each new model trained to correct the mistakes made by its predecessors.

Ensemble Learning Model: Combination Schemes

Workflow: Boosting operates through the following key steps:

Initialization: All training samples are initially assigned equal weights.

Sequential Training: Models are trained one after another.

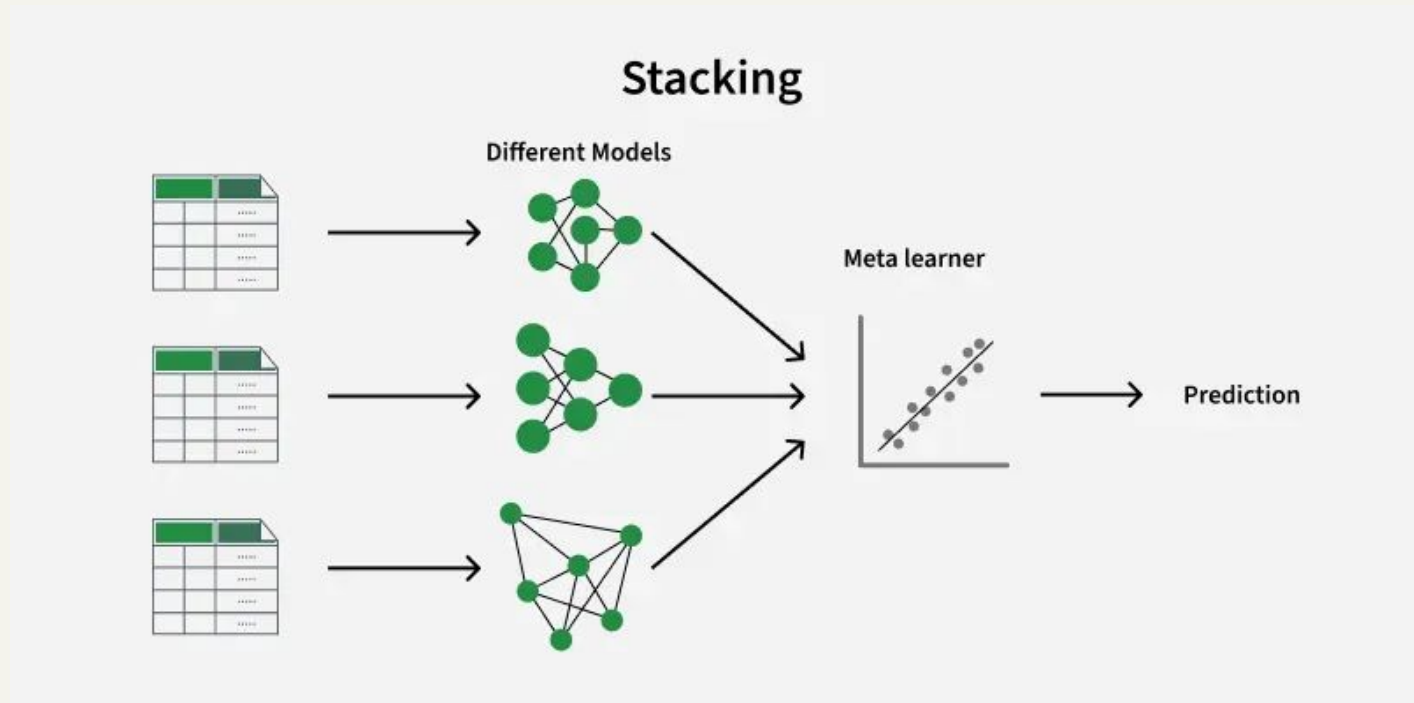
After each round: The model's predictions are compared with the true labels from the training data.

The samples that were misclassified (or had high error) are given higher weights so that the next model pays more attention to them.

Aggregation: After training a set of weak learners, their outputs are combined using a weighted voting (classification) or weighted averaging (regression) mechanism to produce the final prediction.

Ensemble Learning Model: Combination Schemes

Stacking



Ensemble Learning Model: Combination Schemes

What is stacking?

Stacking (or stacked generalization) is an ensemble learning technique that combines multiple models (called base learners) using a meta-model (also called a meta-learner) to improve predictive performance.

Train multiple different models (e.g., Decision Tree, SVM, KNN, etc.) on the same dataset.

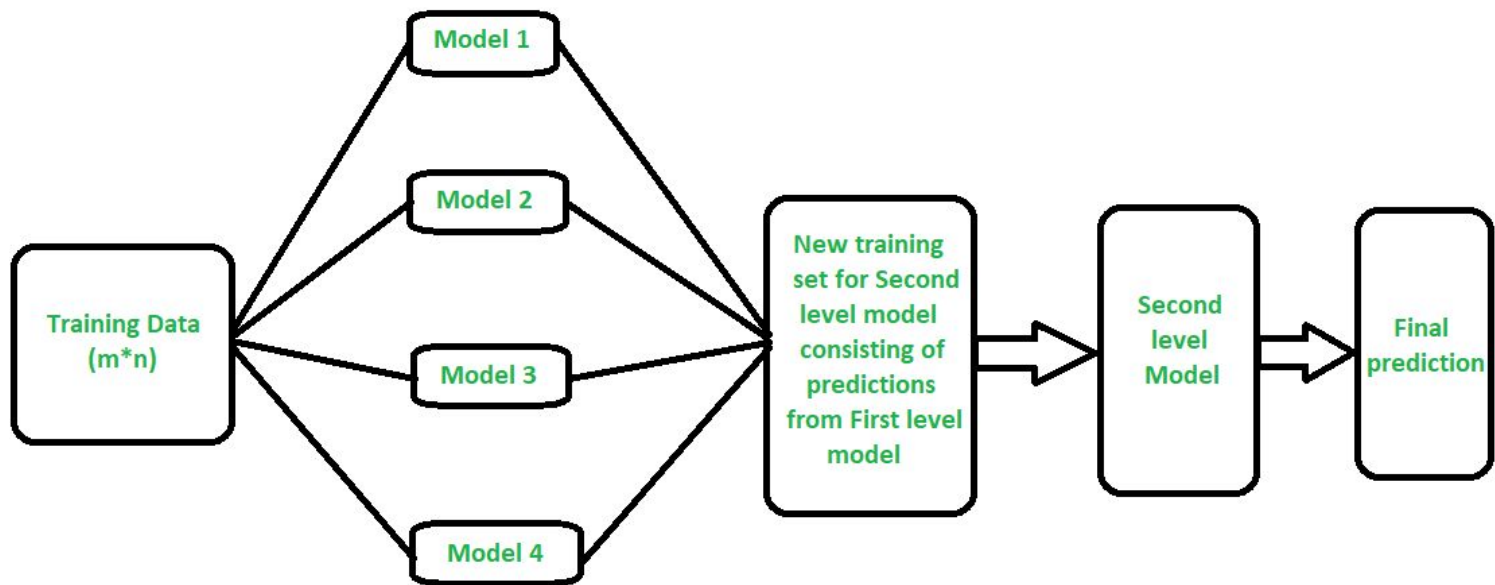
Each model makes predictions on the validation set or new data.

These predictions become inputs to a new model — the meta-learner.

The meta-learner learns to combine these predictions to make the final output.

Ensemble Learning Model: Combination Schemes

Stacking



Ensemble Learning Model: Combination Schemes

Stacking

Feature	Base Models (Level-0 models)	Meta-Model (Level-1 model)
Purpose	Learn patterns from original input features	Learn how to combine predictions from base models
Input	Raw training data (features like $X_1, X_2, \dots, X_n$)	Predictions (or outputs) of base models
Output	Individual predictions (e.g., class probabilities, regression values)	Final prediction (based on base models' outputs)
Training Stage	Trained independently in parallel	Trained after base models, using their predictions
Diversity	Often uses different algorithms (e.g., SVM, RF, KNN)	Usually a simple model (e.g., Logistic Regression, Linear Regression)
Role	Provide different perspectives or decision boundaries	Learns the optimal combination of base model outputs

Ensemble Learning Model: Combination Schemes

Difference between Bagging, Boosting, and Stacking

Feature	Bagging	Boosting	Stacking
Goal	Reduce variance	Reduce bias (and variance)	Combine multiple diverse models
Model Training	Independent and parallel	Sequential, each model improves the last	Parallel base models + one meta-model
Data Sampling	Bootstrapped samples (random subsets)	Full dataset, with sample weights adjusted	Full dataset for each base model
Focus	Averaging predictions to stabilize	Correcting errors of previous models	Learning how to best combine models
Final Prediction	Majority vote (classification) / average (regression)	Weighted vote or sum (boosted)	Meta-model combines outputs of base models
Base Learners	Usually homogeneous (same type, e.g., trees)	Usually homogeneous (e.g., shallow trees)	Often heterogeneous (e.g., tree, SVM, NN)
Risk	Underfitting (if models are too simple) Dr. Shagun Sharma	Overfitting (if not regularized) VIT Bhopal University, Sehore	Overfitting (if improper validation)

07 (b)

Bagging: Random Forest

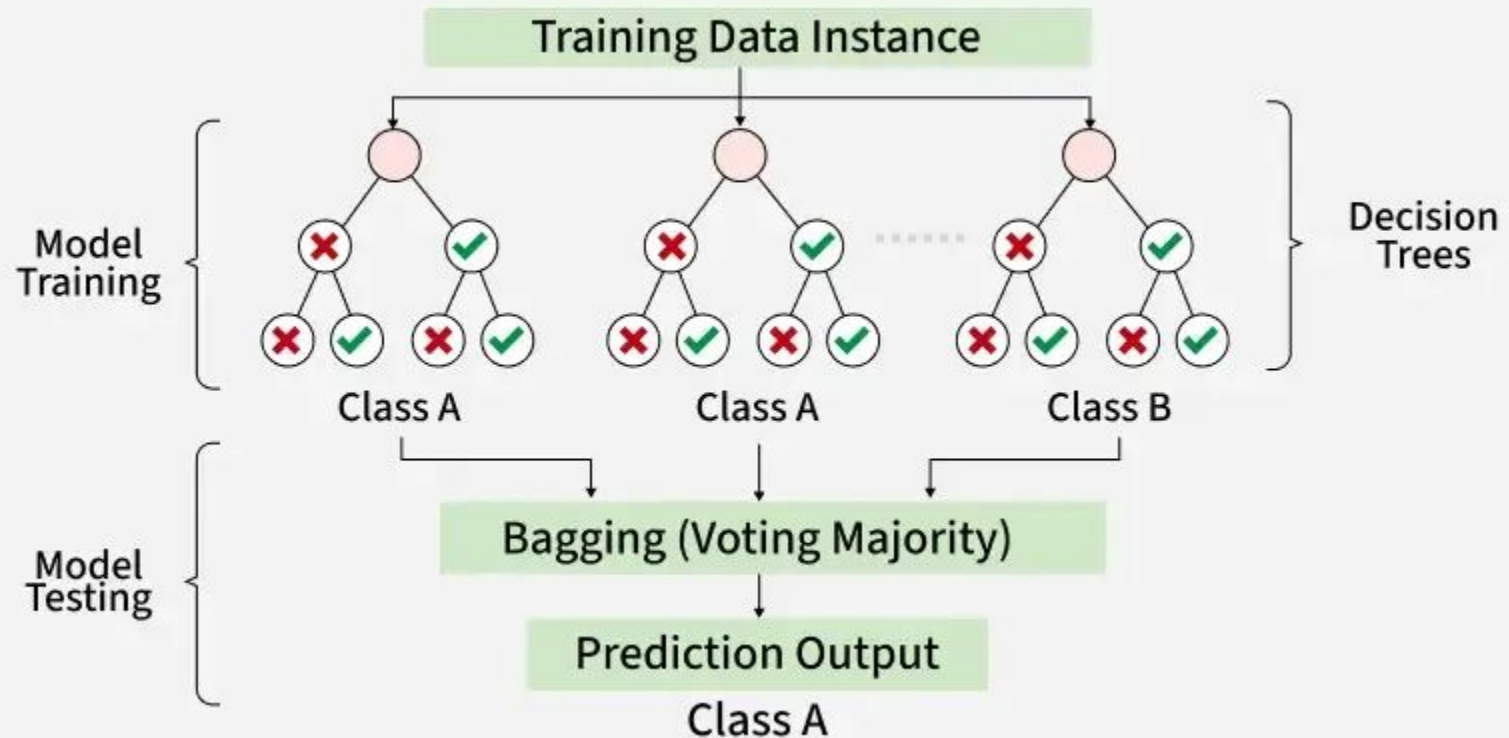
Bagging: Random Forest

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression. This helps in improving accuracy and reducing errors.

Bagging: Random Forest Workflow

- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features:** When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction:** Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions:** For classification we choose a category as the final answer is the one that most trees agree on i.e majority voting.
- For regression we predict a number as the final answer is the average of all the trees predictions.

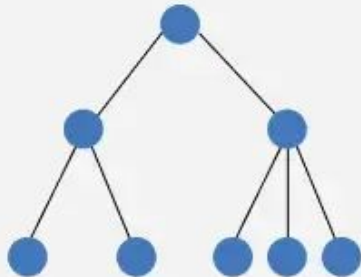
Bagging: Random Forest Workflow



Bagging: Random Forest Workflow

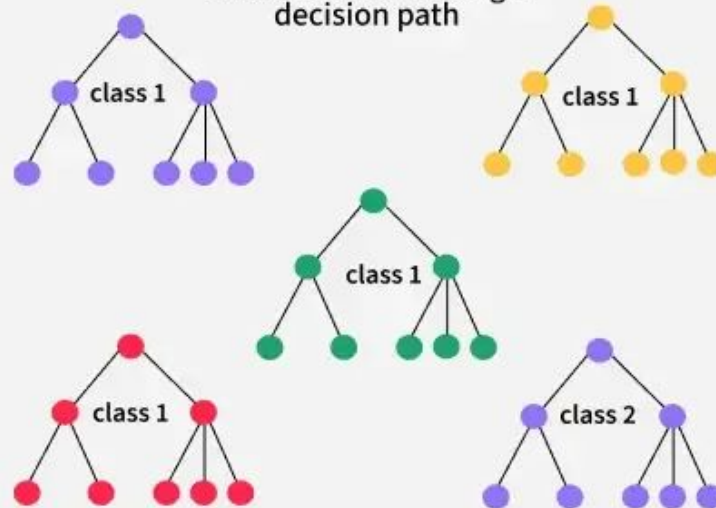
Single Decision Tree

Ensemble of trees for more accurate and robust prediction



Random Forest

Prediction from a single decision path



Bagging: Random Forest (Assumptions of Random Forest)

- **Each tree makes its own decisions:** Every tree in the forest makes its own predictions without relying on others.
- **Random parts of the data are used:** Each tree is built using random samples and features to reduce mistakes.
- **Enough data is needed:** Sufficient data ensures the trees are different and learn unique patterns and variety.
- **Different predictions improve accuracy:** Combining the predictions from different trees leads to a more accurate final result.

08

Clustering- K-Means Clustering